

Efficient Data Storage Systems

(Sistemas de Armazenamento de Dados Eficientes)

Data Reduction and Encryption



Storage Optimizations

Landscape

● Performance

- Caching (*part 1*)
- Prefetching, read-ahead and indexing
- Tiering (*part 1*)
- I/O control policies (*part 1*)
- RAID 0
- Data striping/sharding (*part 3*)

● Space management

- Compression (*part 2*)
- Deduplication (*part 2*)

● Security and privacy

- Encryption (*part 2*)
- Access control
- Trusted execution environments

● Redundancy and crash consistency

- Replication and erasure coding (*part 3*)
- RAID
- Journaling and write-ahead logging
- Checksumming
- **And many more!**

Space Management

Goals and challenges

- **Main goal:** reducing the amount of stored information
 - **Additional storage space available:** data tends to grow...
e.g., backups, AI datasets, analytical workloads, ...
 - **Storage cost reduction:** cost per byte of faster devices is significantly higher
- **Challenges**
 - Achieve good space savings
 - Have a small to no performance penalty on user workloads
 - Do not consume significant system resources
(e.g., RAM, CPU, network, disk)

Data Compression

Brief overview

- **Compression:** Removes redundant content (e.g., bytes) inside and across files
 - Usually used as a static technique
i.e., for sets of files that are not expected to be further updated
- **Lossy compression:** pieces (bits) of the original data are lost during the compression process
 - Useful for media files (e.g., videos, images)
 - Examples: JPEG, MPEG
- **Lossless compression:** original data can be fully recovered (decompressed) from compressed data
 - Useful for storage systems but also other purposes (e.g., network transmission)
 - Examples of tools: Gzip, Winrar, Snappy

Data Compression

Problems

- What if one wants to
 - Eliminate redundancy over a **full disk device**
 - Or **datasets** stored and updated at different times by **distinct users**
 - Or even across **distributed storage nodes**
- Updating data with traditional lossless compression techniques requires decompressing old data, updating it, and compressing it again
 - **Prohibitive performance-wise!**

Data Deduplication

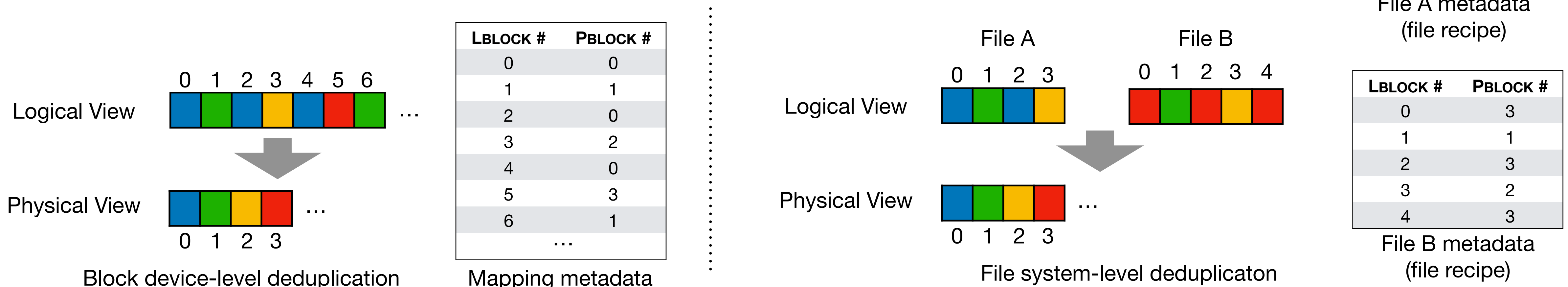
Definition and Goals

- Technique for eliminating coarse-grained and unrelated duplicate data in storage systems
- Goals
 - **Maximize** space savings, also known as **deduplication gain**
 - **Minimize impact** on the system's **performance** and **resource usage** (e.g., CPU, RAM, network, disk)
 - Ensure **data durability** and **crash-consistency**
 - Avoid compromising **data privacy** and **security** (*i.e.*, especially when data from multiple users is being shared)

Data Deduplication

APIs and Abstraction

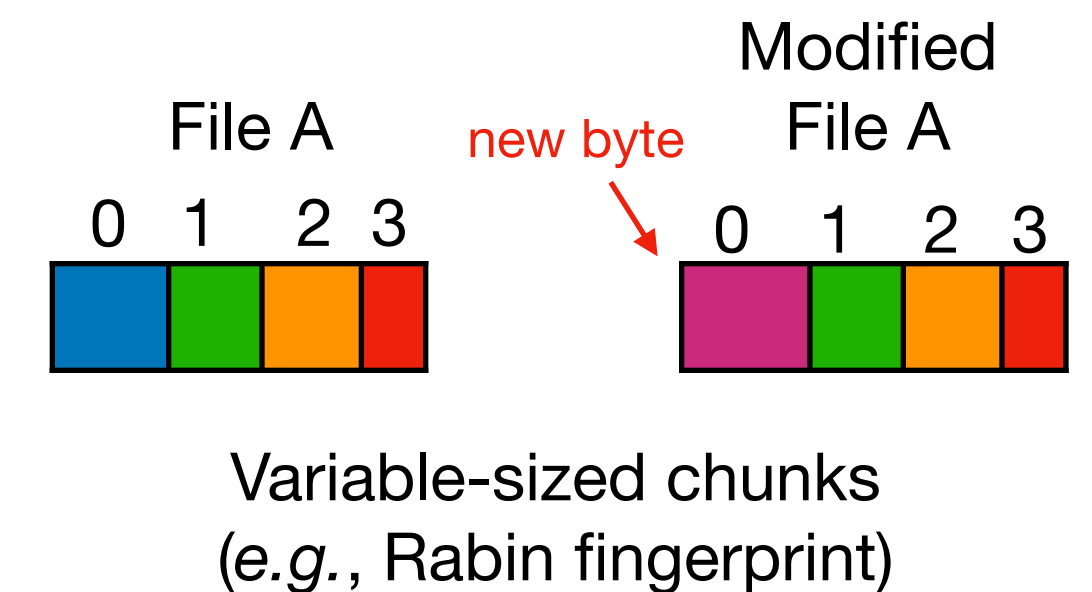
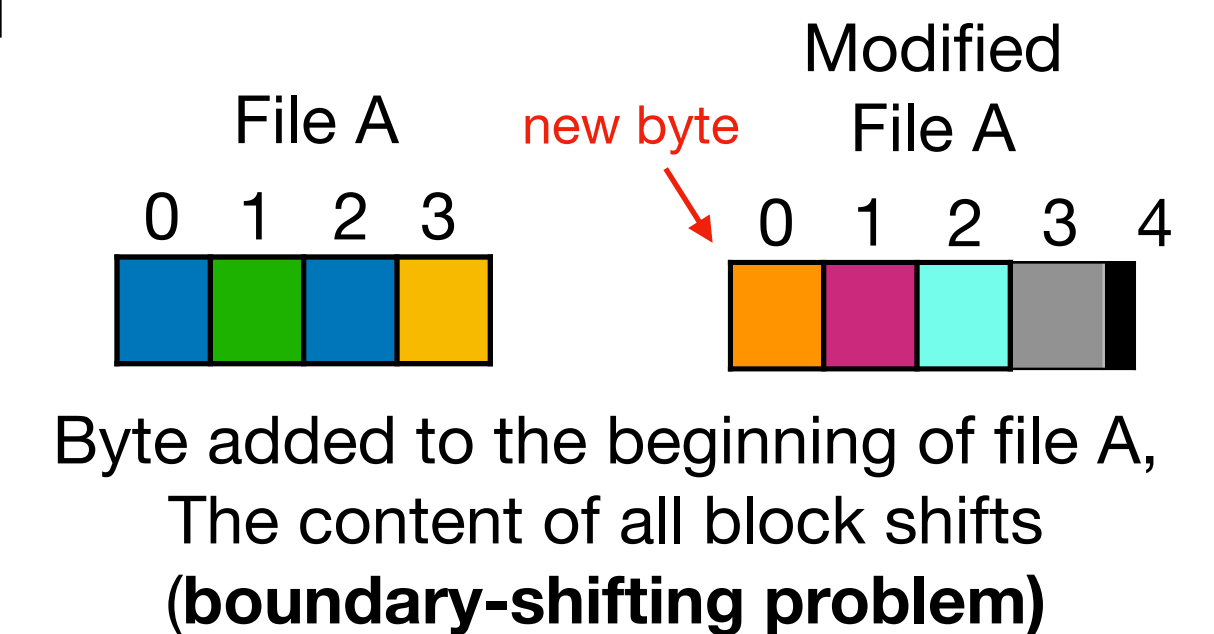
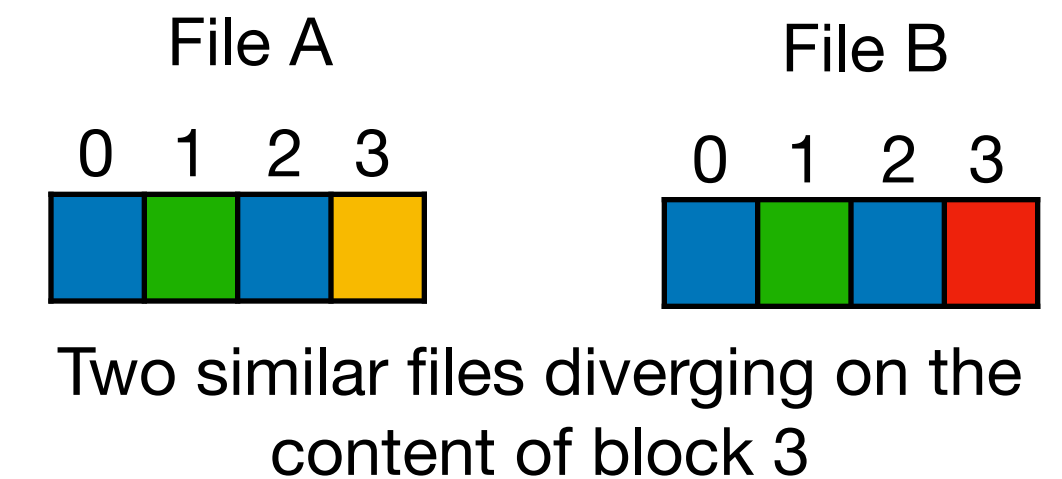
- Deduplication is typically a component of a complex data management system, exposing an API
 - Block device, file system, object store, ... Different APIs have specific challenges, as we will see!
- Bidirectional mapping between two views
 - **Logical view** of data presented to the users/applications
Users see duplicate content and are unaware that it has been removed - **Transparency!**
 - **Physical view** of the device where duplicate content was removed
- **Metadata** is required for this **logical-to-physical** mapping
 - It changes according to the API and deduplication unit



Granularity

At what unit should one perform deduplication?

- The deduplication unit is often referred to as a **chunk**
- **Whole file chunking.** Consider the whole file as the deduplication unit
 - Smaller metadata when compared to the block-based techniques below
 - Reduced deduplication gain, *i.e.*, files diverging on a single byte are not deduplicated
- **Fixed-size chunking.** Data is partitioned into chunks of the same size (*e.g.*, 4KB)
 - Larger metadata when compared to whole-file chunking
 - Better deduplication gain, *i.e.*, deduplication is possible even if files are not identical
 - Suffers from the **boundary-shifting problem** (for chunks calculated from files)
- **Variable-sized chunking** (aka **content-defined chunking**). Data is partitioned into variable-sized chunks. The boundaries of chunks are set by the content itself (*i.e.*, a pattern; see the Rabin Fingerprint scheme)
 - Similar metadata size to fixed-size chunking, but requires extra computation
 - Avoids the boundary-shifting problem, improving further deduplication gain

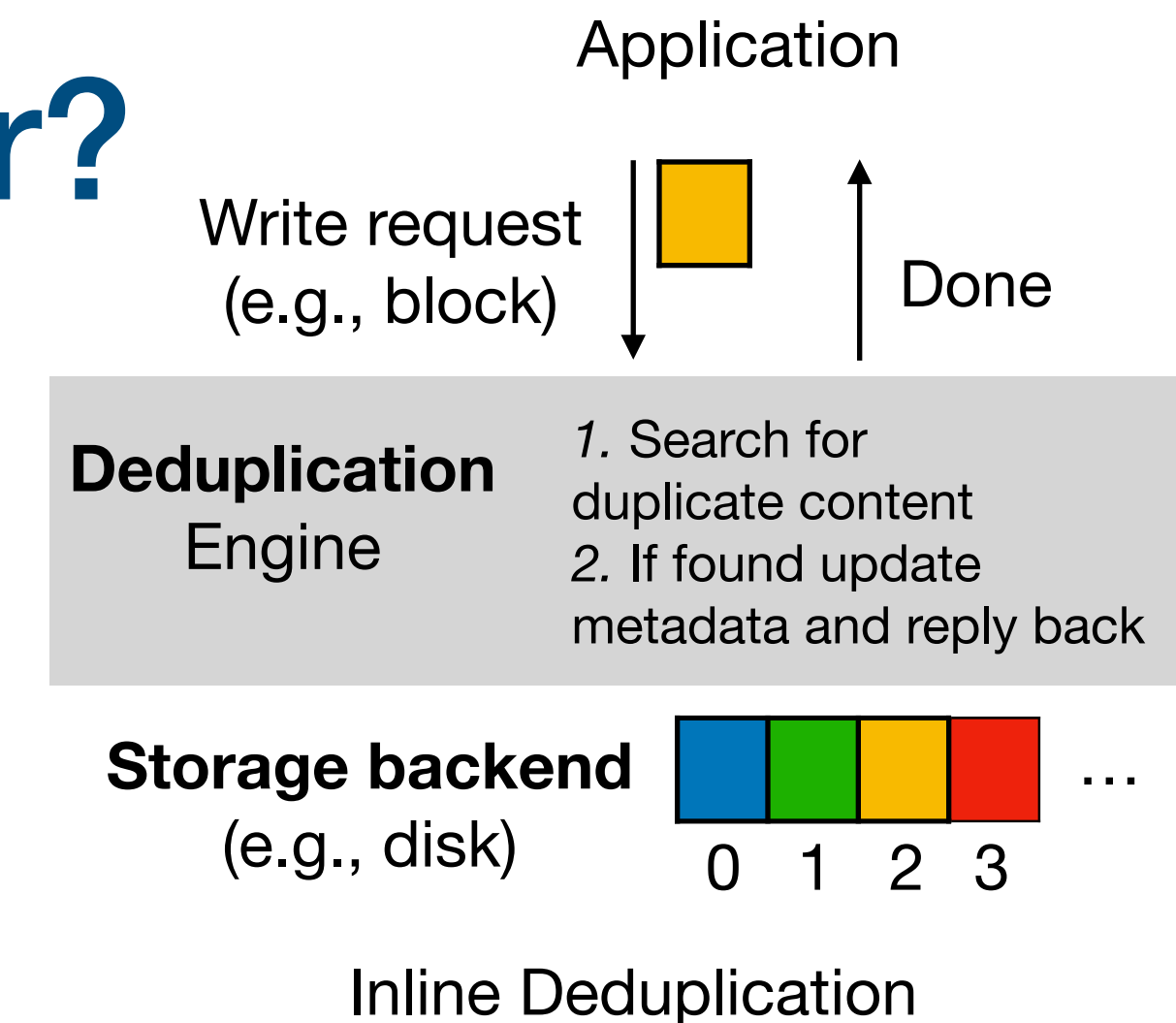


Timing

When should deduplication occur?

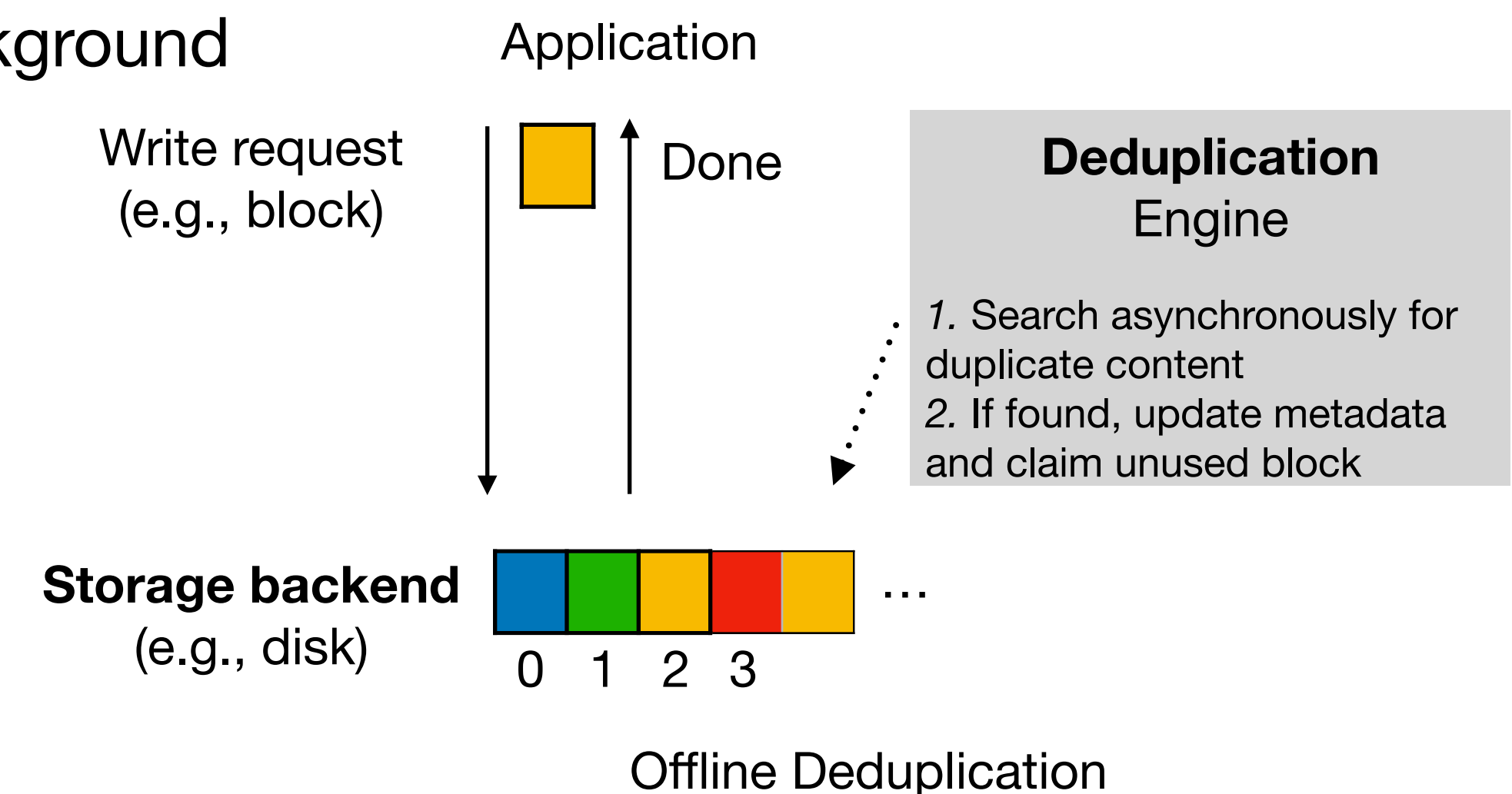
◎ **Inline Deduplication.** Duplicate chunks are identified before being written to the storage backend

- Duplicates are never written to the storage backend
Avoids writing duplicate content in advance!
- Write requests must be intercepted by the deduplication engine
Extra processing in the critical I/O path!



◎ **Offline Deduplication.** Duplicate chunks are identified in the background

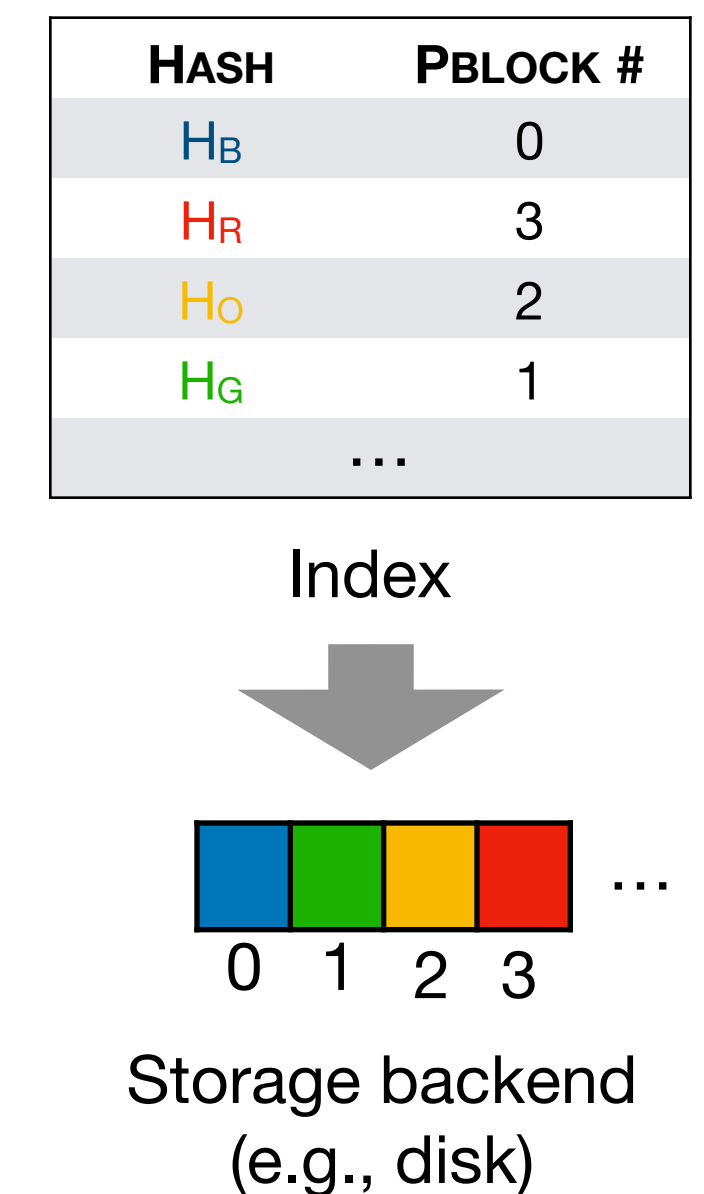
- Chunks are first written into the storage backend
Updated chunks are marked as dirty (e.g., in a bitmap structure)
Small overhead in the critical I/O path!
- Redundant chunks are identified asynchronously (e.g., by a background deduplication process/thread)
Redundant chunks are held at the backend until deduplication kicks off!



Indexing

How to search for duplicates efficiently

- Scanning the storage backend to find duplicate chunks is inefficient!
 - **Solution:** use an index to map the content of unique chunks to their location at the backend (e.g., physical block number)
This structure is required by both inline and offline deduplication!
- A hash sum (signature) can represent the content of each unique chunk
 - **Efficient** space-wise, but requires extra computation (*i.e.*, CPU)
 - One must be careful with hash **collisions**, *i.e.*, use a strong **cryptographic hash** (e.g., sha-512) or do a **byte-wise** comparison of potential duplicate chunks
- Where should the index reside?
 - **Persistent storage.** Good for faster recovery time upon failures-
Bad performance-wise, especially for inline deduplication
 - **Main memory.** Good for access speed, but what if the index size is too large?



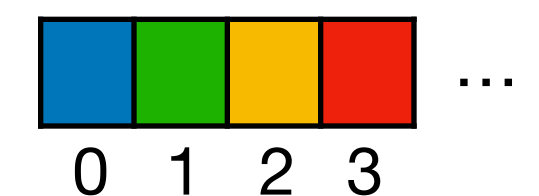
Indexing

Know your workload

- With 4KB (2^{12}) chunks, a system with 1TB (2^{40}) can persist $2^{40}/2^{12}$ (268 435 456) chunks
 - Assuming 72 bytes per index entry (64-byte hash and 8-byte block number), in the **worst case** (all chunks are unique), the index requires up to **18 GiBs** of memory
- **Partial index.** A subset of unique chunks is indexed and, thus, deduplicated
 - It works as a cache and can have an eviction policy
 - LRU policies explore the **temporal locality** of workloads, *i.e.*, duplicate chunks are expected to appear several times in a short time window
- **Sparse index.** Each index entry maps to a group of contiguous stored chunks
 - When a duplicate chunk is identified, the hashes of all chunks belonging to the same group are brought to memory from the disk
 - Explores **spatial locality** of workloads, *i.e.*, chunks stored in a given data stream are expected to appear in subsequent streams
 - **Note:** there are sparse indexes based, for instance, on the content similarity of groups of chunks

HASH	PBLOCK #
H_B	0
H_R	3

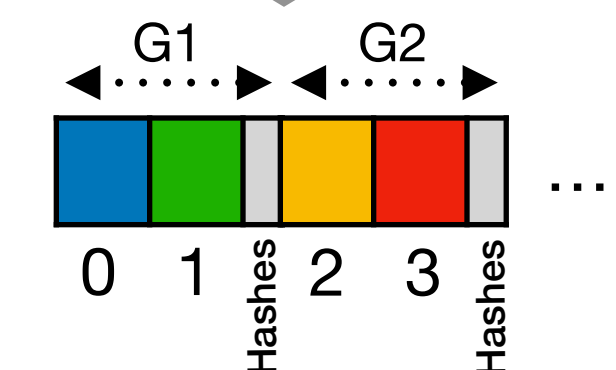
Partial Index



Storage backend

HASH	PBLOCK #
H_{G1}	0
H_{G2}	2
...	

Sparse Index



Storage backend

Modifying Shared Data

Deleting and updating data

● What if logical chunks are **deleted** (or zeroed)?

- Deleting a shared chunk can lead to **data loss**!
- *Example:* If one zeroes physical block 0, then logical blocks 2 and 4 become corrupted

● The **number of references** to unique chunks must be kept (e.g., at the index)

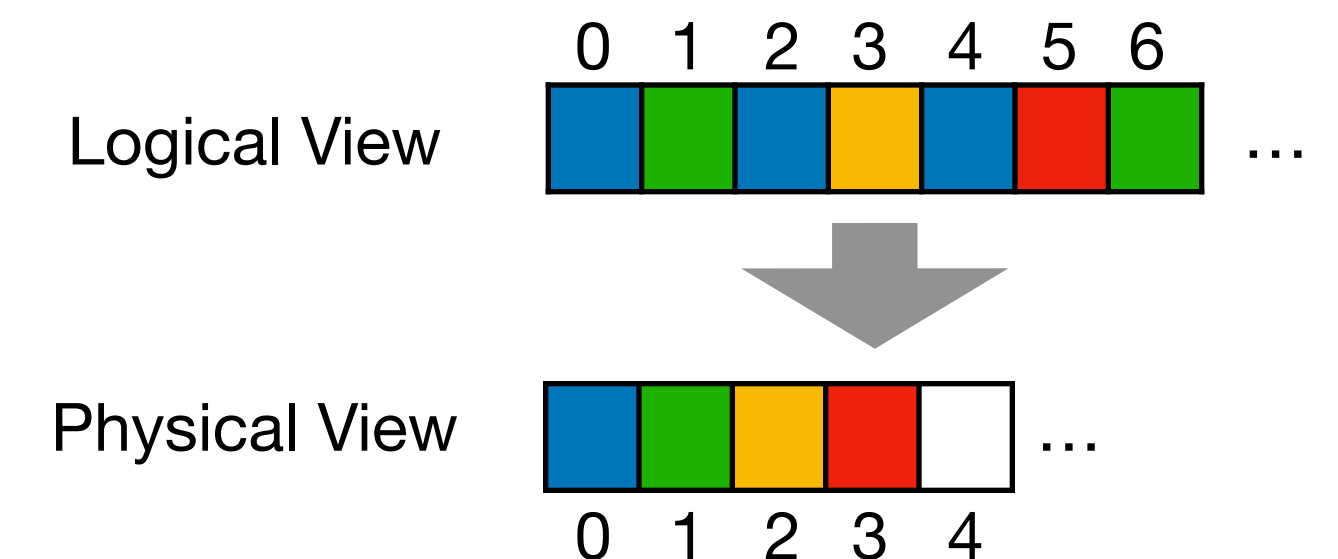
- References are incremented when new duplicates are found and decremented when logical blocks are erased (or updated)

LBLOCK #	PBLOCK #
0	0
1	1
2	0
3	2
4	0
5	3
6	1
...	

Mapping metadata

HASH	PBLOCK	REF
H _B	0	3
H _R	3	1
H _O	2	1
H _G	1	2
...		

Index



Block device-level deduplication

Modifying Shared Data

CoW and free space management

● What if logical chunks are **updated**?

- Besides doing reference counting, if the physical block is shared, the new content must be written to an **unused** block (**Copied-on-Write**)
- *Example:* If logical block 0 is updated, the new content must be written at physical block 4

LBLOCK #	PBLOCK #
0	0
1	1
2	0
3	2
4	0
5	3
6	1
...	

Mapping metadata

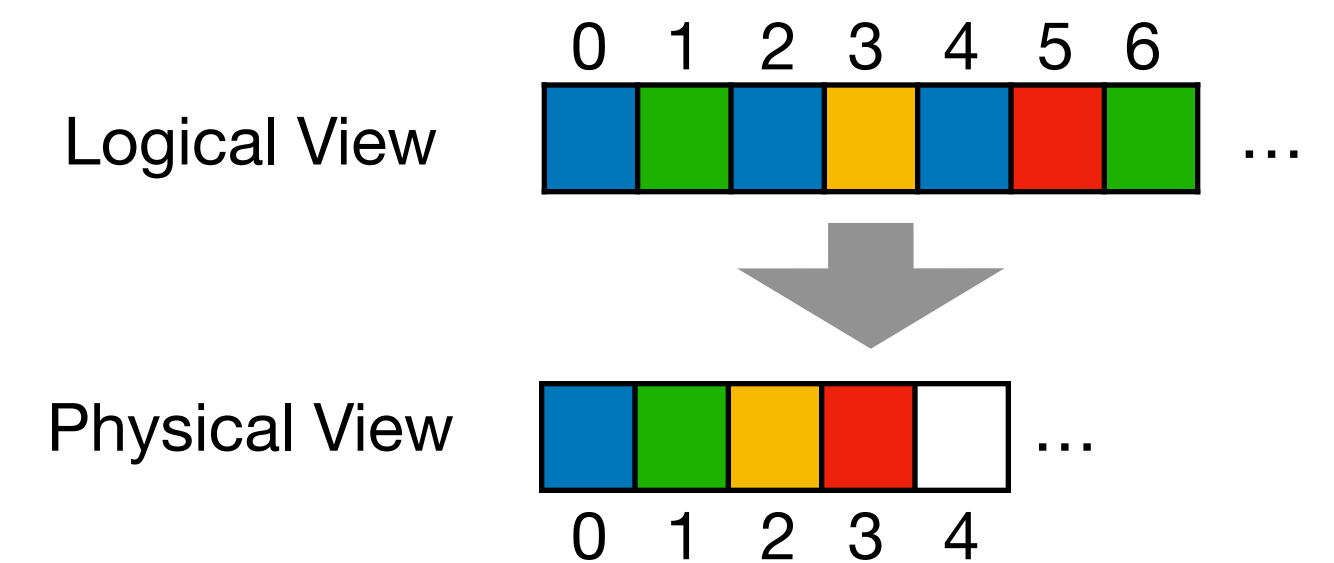
HASH	PBLOCK	REF
H _B	0	3
H _R	3	1
H _O	2	1
H _G	1	2
...		

Index

● **Free Space Management** and **Garbage Collection** for providing free blocks for **CoW** and freeing physical blocks with zero references are required

● Deleting and updating data is tricky!

- Multiple metadata structures are updated and must be coherent under failures (**crash-consistency**)
- With offline deduplication, different threads handle write requests and do deduplication (**concurrency issues**)

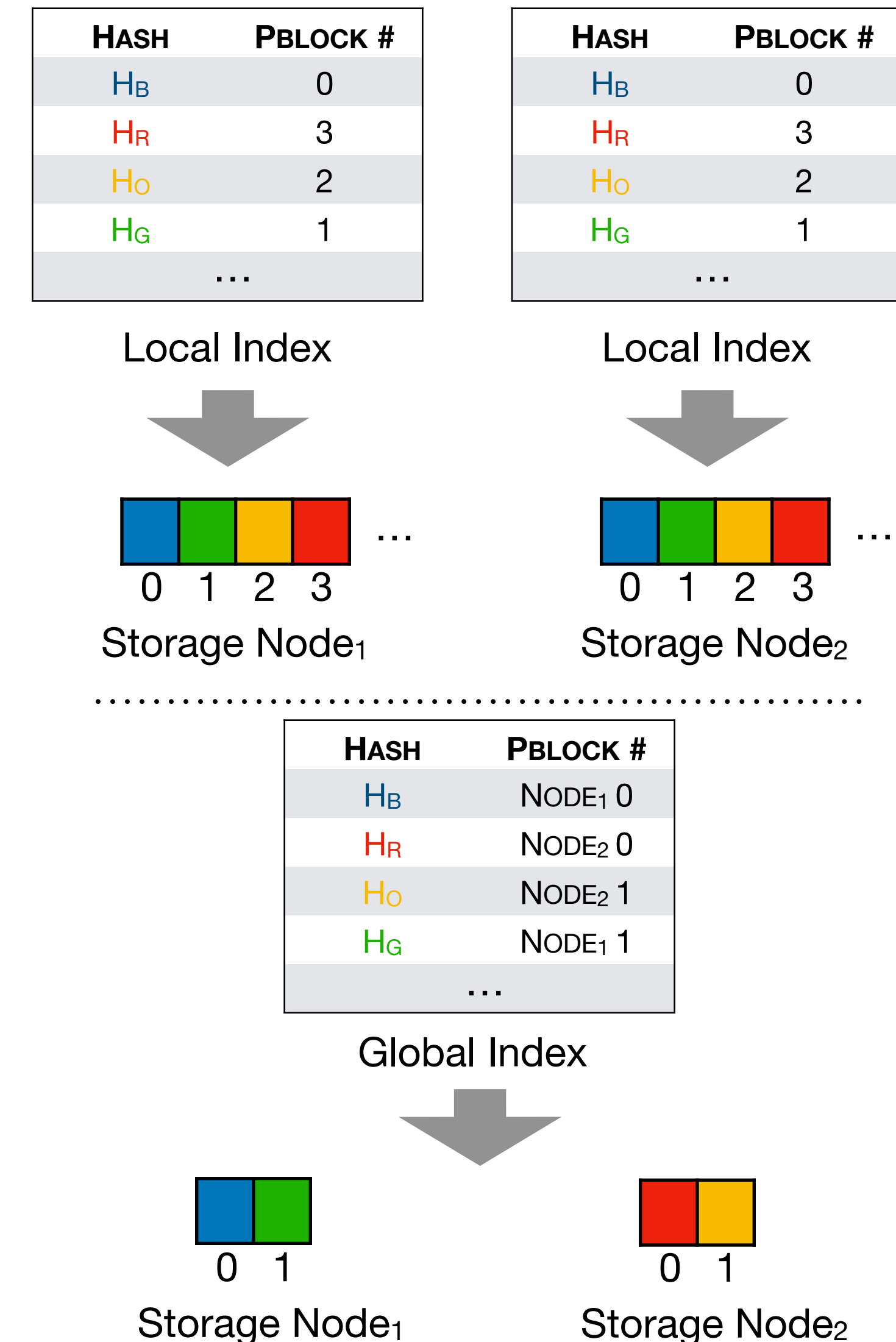


Block device-level deduplication

Deduplication

Leveling up

- ◎ **Distributed Deduplication.** Eliminating redundant data across distributed storage nodes
 - Should each node find duplicates only at its local storage?
 - **Local indexes** have worse deduplication gain
 - Should this be done globally across the distributed system?
 - **Global indexes** require coordination and scalability
- ◎ Deduplication can be combined with other space-saving techniques
 - **Delta-encoding.** Store only the deltas of similar (but not identical) chunks
 - **Compression.** Compress unique chunks at the storage backend
- ◎ Deduplication designs must be tuned according to the hardware characteristics
 - NVMe, SATA SSDs, HDDs
 - Available memory, computational power (e.g., for hash calculation)



Data Security

Brief overview

- **Data Encryption:** Privacy protection for sensitive data
 - **Encryption at rest:** Data is encrypted before being stored persistently
 - **Encryption in transit:** Data is encrypted before being sent through the network
 - **Encryption in use:** Data is encrypted while being processed
- **Access Control:** Avoid unauthorized access to users' data

Storage Encryption

Challenges

- **Data at rest** should remain **private** (confidential) against strong adversaries
 - Can read stored encrypted data multiple times
 - Can ask the storage system to encrypt and store their own plaintexts
 - Can manipulate encrypted content
(e.g., move to a different location on the disk and ask for decryption)
- Writing and reading data must remain **efficient**
 - Encryption/Decryption operations are fast
 - Storage space is not wasted by having ciphertexts larger than the original plaintext

Disk Encryption Modes

AES XTS

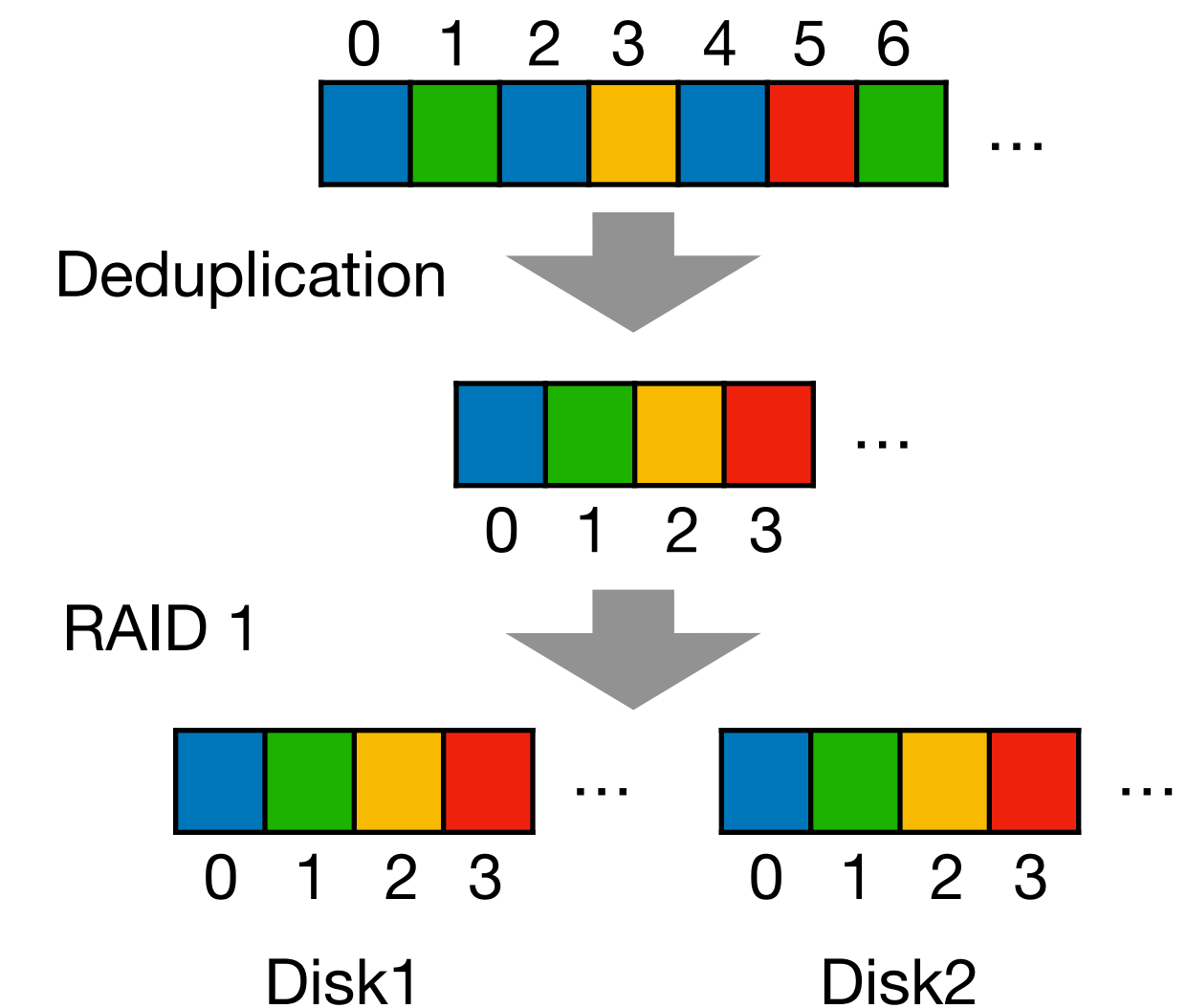
- Traditional modes such as **AES Cipher-block chaining** (CBC) and **Counter** (CTR) are typically used for **secure data transmission**
 - They require unique initial states (initialization vector - *IV*) and/or padding that lead to storing extra data within/along the ciphertext
i.e., these are not **length-preserving**
- **AES XTS** is based on the concept of a **tweakable block cipher**
 - It produces unique ciphertexts (even when given identical plaintexts) without the use of *IVs*
 - It does not require chaining (unlike CBC), similar to CTR (good parallelization)
 - It is length-preserving!
- XTS **does not provide integrity guarantees**; for that authenticated encryption AES-GCM (or other methods) must be used

Using Deduplication with Data availability and encryption techniques

- When combined with **redundancy techniques** (e.g., RAID, replication), when should deduplication be applied? Before or after these?

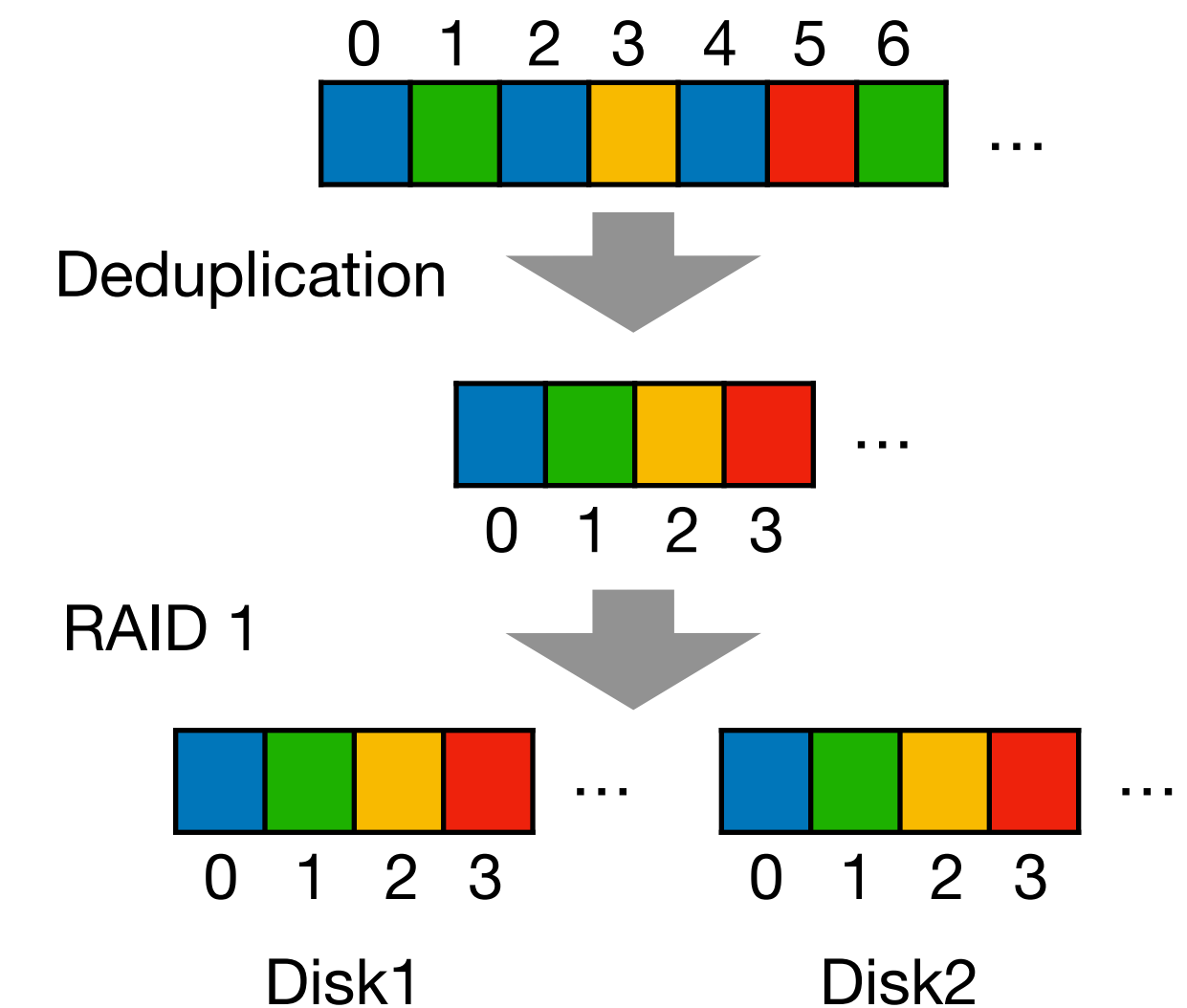
Using Deduplication with Data availability and encryption techniques

- When combined with **redundancy techniques** (e.g., RAID, replication), when should deduplication be applied? Before or after these?
 - **Before**, first identify unique data and then replicate for fault tolerance
- What about when combined with **data encryption**?



Using Deduplication with Data availability and encryption techniques

- When combined with **redundancy techniques** (e.g., RAID, replication), when should deduplication be applied? Before or after these?
 - **Before**, first identify unique data and then replicate for fault tolerance
- What about when combined with **data encryption**?
- What if one wants to **encrypt data at the client** premises and perform deduplication at an untrusted server (e.g., cloud computing service)?
 - We need either **deterministic encryption** (content-preserving) techniques or **trusted execution environments**¹



¹ Miranda M, Esteves T, Portela B, Paulo J. **S2Dedup: SGX-enabled Secure Deduplication**. ACM International Systems and Storage Conference (SYSTOR)

More Information

- **A Survey and Classification of Storage Deduplication Systems** - João Paulo and José Pereira, ACM CSUR, 2014
- **A Comprehensive Study of the Past, Present, and Future of Data Deduplication** - Wen Xia et al, IEEE, 2016
- **S2Dedup: SGX-enabled Secure Deduplication** - Miranda M, Esteves T, Portela B, Paulo J. ACM SYSTOR, 2021
- **Chapter 7** - William Stallings. **Cryptography and Network Security**. Global Edition, Eight Edition.

Questions?